



线程

Threads



主要内容

- 线程概念 Thread concept
- 多核编程 Multicore programming
- 多线程模型 Multithread model
- 线程库 Thread library



进程及存在的问题

- 操作系统中引入进程的目的，是为了描述和实现多个程序的并发执行，以改善资源利用率以及提高系统吞吐量。
- 进程的两个基本属性
 - 进程是一个可拥有资源的独立单位：为了能够独立运行，进程必须拥有一定的资源，如内存空间、I/O设备等。这些资源使得进程能够执行其任务，并在需要时与其他进程进行交互。
 - 进程同时又是一个可独立调度和分派的基本单位，使之成为一个能独立运行的基本单位，
- 这两个属性构成了进程并发执行的基础。



进程及存在的问题

- 由于进程是一个资源的拥有者，因而在创建、撤消和切换中，系统必须为之付出较大的时间和空间开销。
- 为使程序能并发执行，系统必须进行以下的一系列操作：
 - (1) 创建进程，系统在创建一个进程时，必须为它分配其所必需的、除处理机以外的各种资源，如内存空间、I/O设备，以及建立相应的PCB；
 - (2) 撤消进程，系统在撤消进程时，又必须先对其所占有的资源执行回收操作，然后再撤消PCB；
 - (3) 进程切换，对进程进行上下文切换时，需要保留当前进程的CPU环境，设置新选中进程的CPU环境，因而须花费不少的处理机时间。
- 所以，在系统中所设置的进程，其数目不宜过多，进程切换的频率也不宜过高，这也就限制了并发程度的进一步提高。

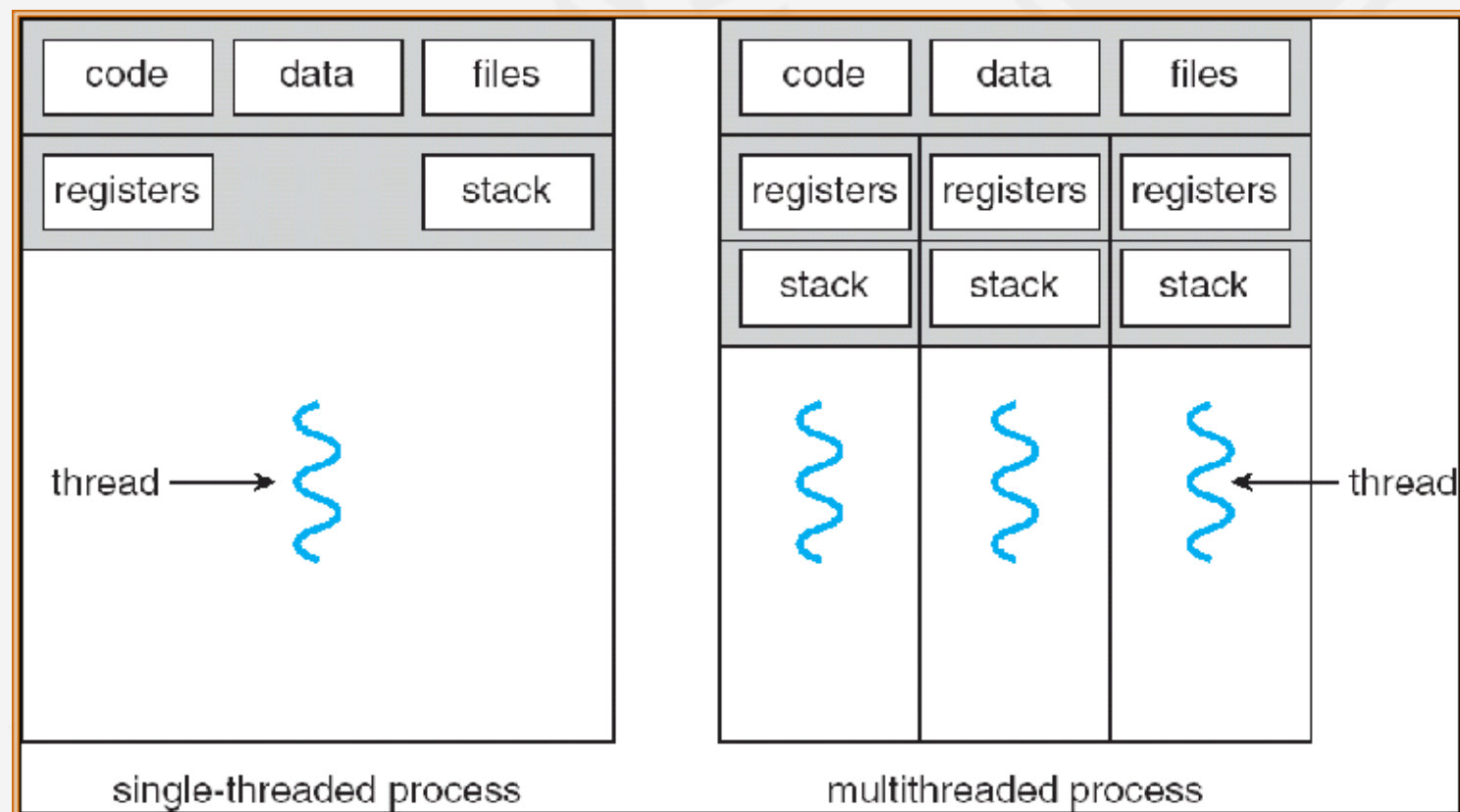


线程的引入

- 为了更好地实现并发执行的同时尽量减少系统的开销，操作系统引入线程的概念。
- 线程设计思想：将进程的上述两个属性分开，由操作系统分开处理。
 - 线程，作为调度和分派的基本单位，不作为拥有资源的单位，这样切换速度快，代价小。
 - 进程，仍然保留其作为拥有资源的基本单位，对之不进行频繁的切换。
- 由于线程具有许多传统进程所具有的特征，所以又称之为轻量级进程（Light-weight Process）

线程概述

- 线程是CPU运行的一个基本单元，包括
- 程序计数器 (program counter)
- 寄存器组 (register set)
- 堆栈空间 (stack space)
- 一个线程与它的对等线程共享：
 - 代码段 (code section)
 - 数据段 (data section)
 - 操作系统资源 (operating-system resources)



- 传统的进程等价于只有一个线程的任务

线程概述

- 下图左侧列出进程中的所有线程共享的一些项目。
- 下图右侧列出每个线程专用的一些项目。

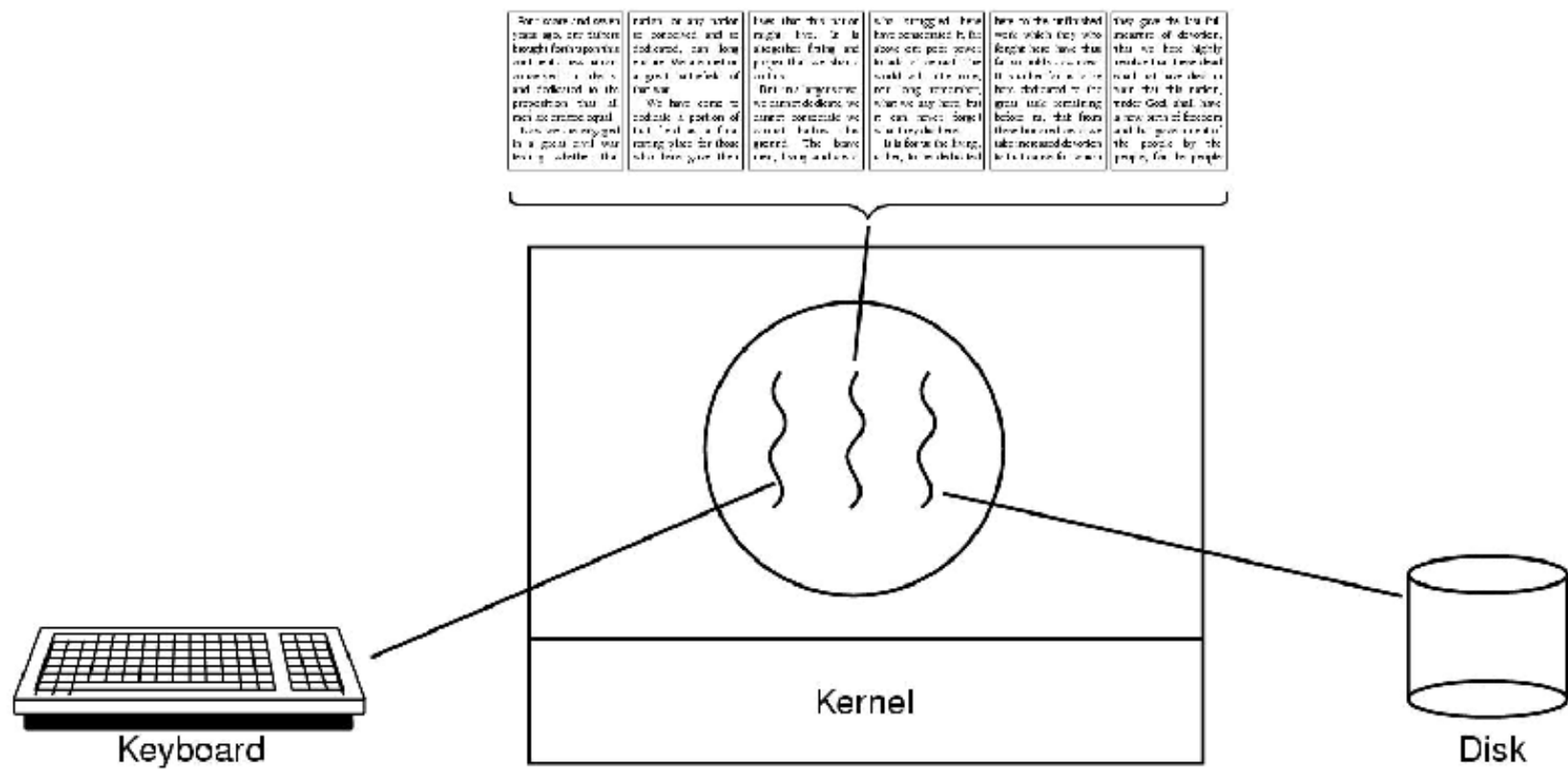
Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

线程的优点

- 响应性 (Responsiveness)
- 如果进程的某个部分被阻塞，其他线程可能会允许继续执行，这对用户交互任务尤其重要
- 资源共享 (Resource Sharing)
- 内存和资源共享、代码共享（运行一个进程内多个线程执行一个相同的代码），共享比进程容易
- 经济 (Economy)
- 进程创建和切换比较耗费资源，而线程允许资源共享，创建和切换线程更加经济
- 伸缩性 (Scalability)
- 充分利用多处理器体系架构

多线程的应用场景

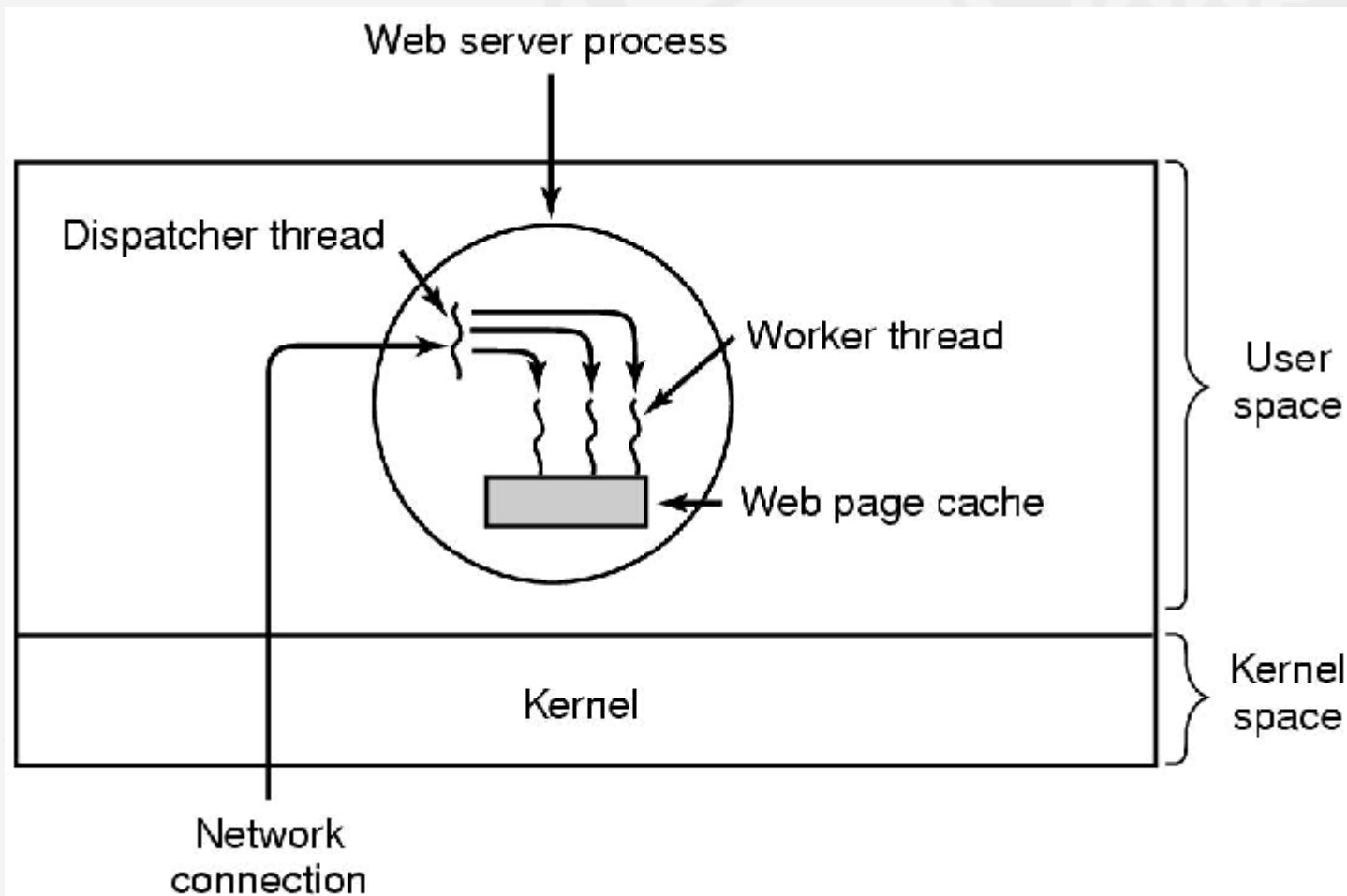
- 一个拥有三个线程的字处理程序
- 三个线程分别负责：接收输入、内容排版和存储到硬盘。
- 三个线程互不影响，可以更好的响应用户。





多线程的应用场景

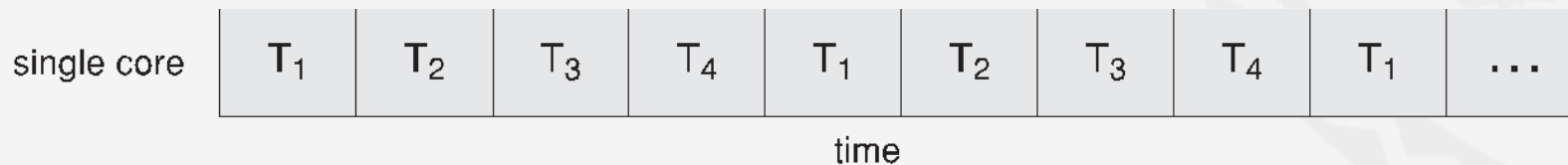
- 一个多线程的Web Server
- 包括一个任务分派器 (Dispatcher) 和多个工作 (worker) 线程。
- 多个工作线程可以共享web page cache, 数据共享比多进程之间更容易。



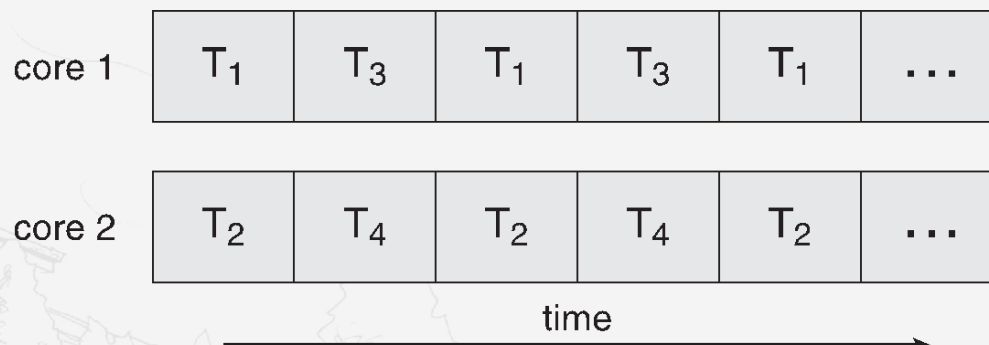
多核编程 (Multicore Programming)

■ 并发(Concurrency) 和 并行(Parallelism)

■ 并发：通过调度器，支持多个任务在单处理器/单核系统上交替执行



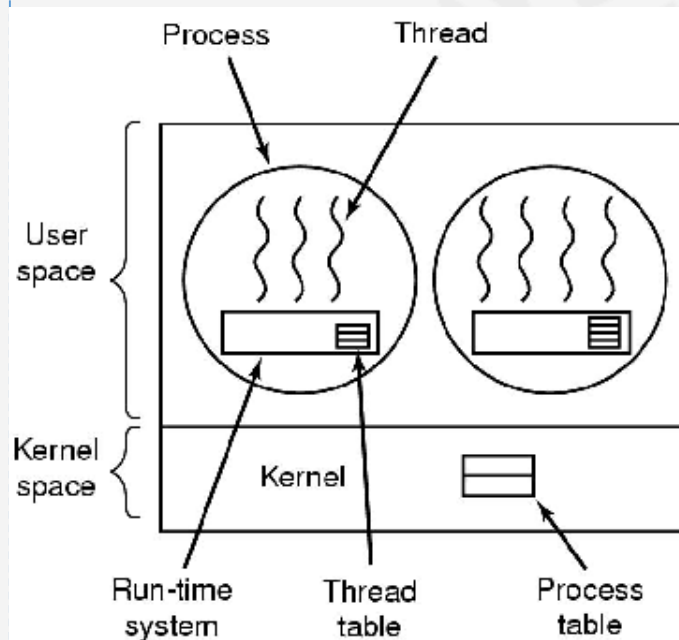
■ 并行：在多处理器 (multiprocessors) / 多核 (multicore) 的环境下，同时真正同时执行多个任务



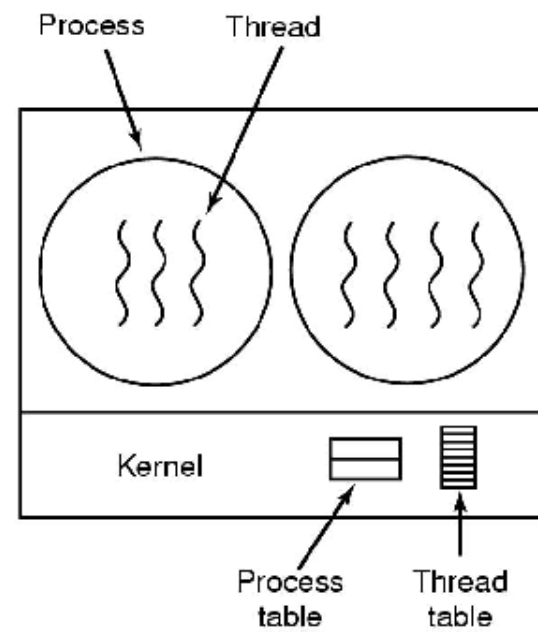
线程的实现

■ **用户级线程 (User level thread)** 利用用户级的线程库进行管理。线程库提供对线程的创建、调度和管理。内核不知道用户线程的存在。因此，任何一个用户级线程执行阻塞原语的时候就会引起整个进程阻塞，即使当时还有该进程内还有其他的线程可以被调度执行。

■ **内核级线程 (Kernel level thread)**：操作系统内核负责其创建、调度和管理。内核直接管理线程，因此当一个线程调用阻塞原语，进入阻塞态时，内核能调度当前进程内的其他线程执行。



(a) 用户级线程



(b) 内核级线程

线程的实现

■ 用户级线程 User level thread 优点

■ 1. 可以在不支持线程的操作系统中实现

■ 2. 创建和销毁线程、线程切换代价等线程管理的代价比内核线程少得多

■ 3. 允许每个进程定制自己的调度算法，线程管理比较灵活。

■ 4. 线程的调度不需要内核直接参与，控制简单。

■ 用户级线程 User level thread 缺点

■ 1. 线程发生I/O或页面故障引起的阻塞时，会阻塞整个进程

■ 2. 一个单独的进程内部，没有时钟中断，所以不可能用轮转调度的方式调度线程

■ 内核级线程 Kernel level thread 优点

■ 1. 当有多个处理机时，一个进程的多个线程可以同时执行。

■ 2. 如果进程中的一个线程被阻塞，能够切换同一进程内的其他线程继续执行

■ 3. 所有能够阻塞线程的调用都以系统调用的形式实现

■ 内核级线程 Kernel level thread 缺点

■ 1. 代价大，性能较差

■ 2. 内核统一调度算法，进程不能有定制算法

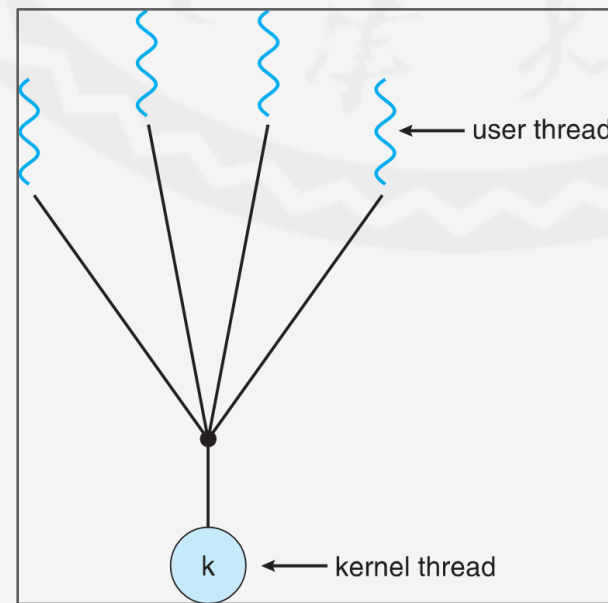
■ 3. 调度由内核完成，需要内核支持线程（旧操作系统可能不支持），控制较复杂。

多线程模型

- 用户空间中的代码被限制了只能使用一个局部的内存空间，我们说这些程序在用户态（User Mode）执行。内核空间中的代码可以访问所有内存，我们称这些程序在内核态（Kernal Mode）执行。
- 现代操作系统，如 Windows, Linux, Mac OS X和 Solaris等，都支持用户级线程和内核级线程。
- 用户级线程由线程库创建和管理，而操作系统只能管理内核级线程。二者需要映射。
- 内核级线程和用户级线程之间的映射关系：多对一模型、一对一模型和多对多模型。

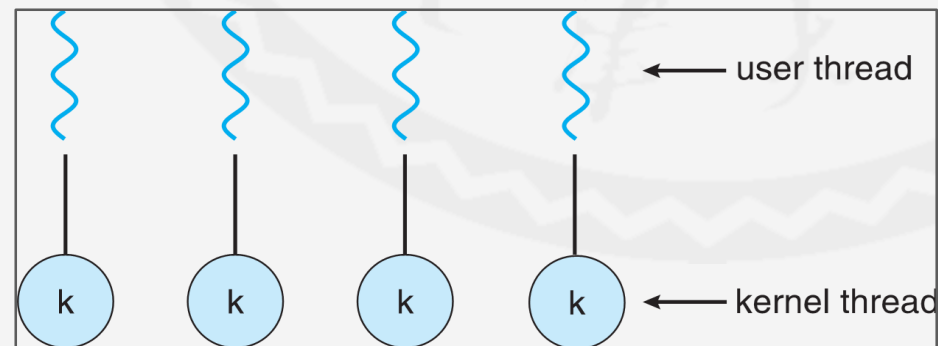
多线程模型

- **多对一模型 (Many-to-One Model)**：这意味着所有的用户级线程都必须共享同一个内核级线程，从而无法在多核处理器上并行执行。如果一个用户级线程进行了阻塞操作，整个进程都会被阻塞。
 - 现代操作系统较少使用，因为无法利用多个处理核并行执行。
 - 任一时刻只有一个线程访问内核，并行性差。
 - 线程管理由线程库在用户空间中进行，效率比较高



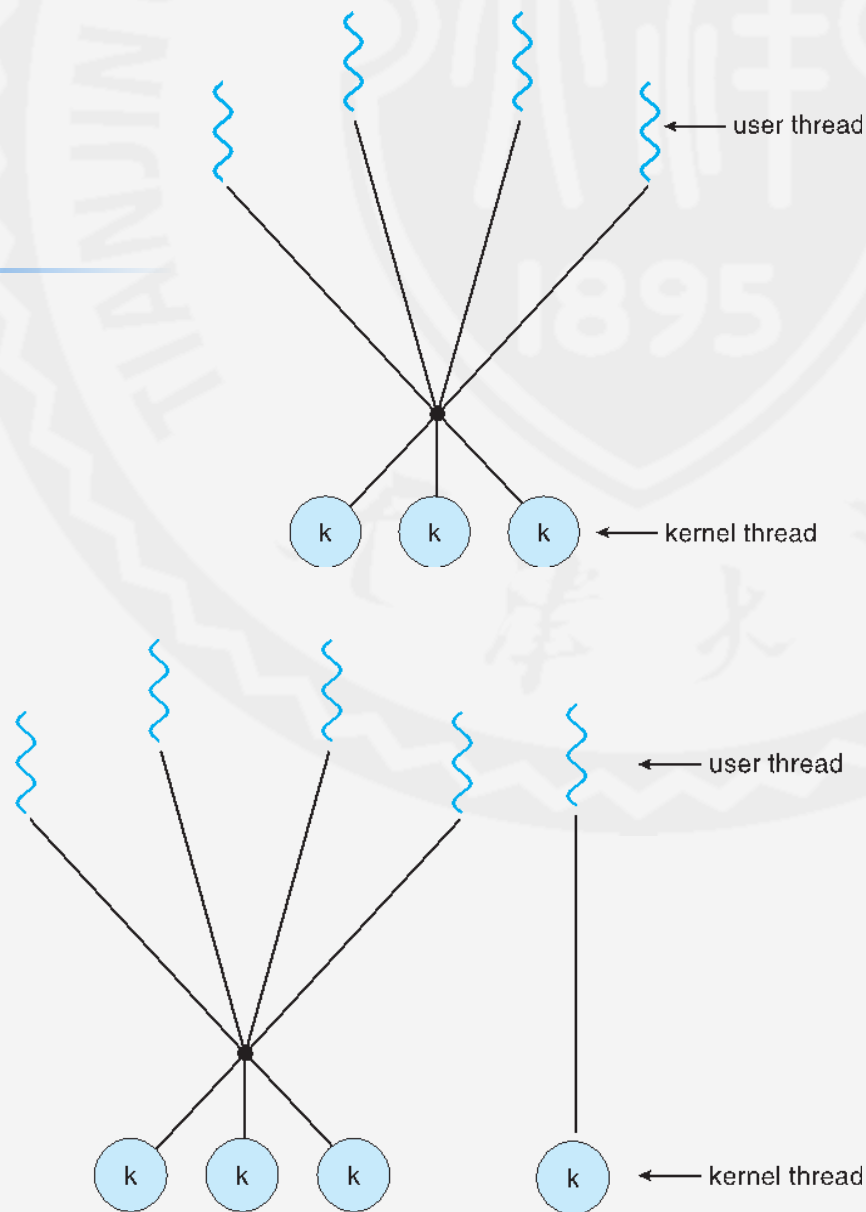
多线程模型

- **一对一模型 (One-to-One Model)**：每个用户级线程都映射到一个独立的内核级线程。这种模型允许线程在多核处理器上并行执行，并且一个线程的阻塞不会影响到其他线程。然而，由于每个用户级线程都需要一个对应的内核级线程，这可能会导致较大的系统开销。
- Linux 及 Windows 家族都实现了一对一模型。



多线程模型

- **多对多模型 (Many-to-Many Model)**：多个用户级线程 (n 个) 可以被映射到数量相同或更少的内核级线程 (m 个) ($n \geq m$)。这种模型结合了对一模型和一对一模型的优点，允许开发者创建大量的用户级线程，同时保持系统开销在合理的范围内。多对多模型还可以在一个线程阻塞时，由内核调度另一个线程来执行，从而提高了系统的并发性能。
- **双层模型 (Two-Level Model)**：允许多对多模式和一对一模式并存。即仍然多路复用多个用户级线程到同样数量或更少数量的内核线程，也允许绑定某个用户线程到一个内核线程。

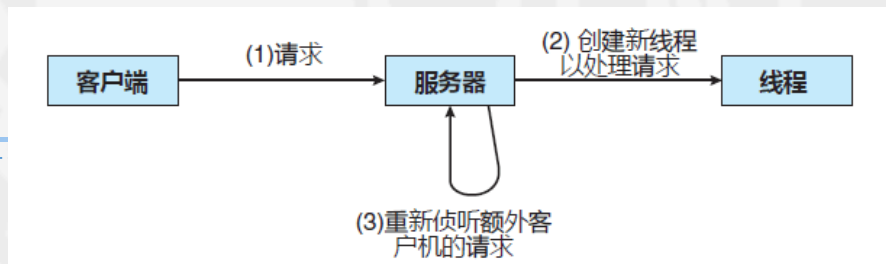


线程库 (Thread Library)

- 线程库为程序员提供创建和管理线程的 API。实现线程库的主要方法有两种：
 - 在用户空间中提供一个没有内核支持的库。这种库的所有代码和数据结构都位于用户空间。这意味着，调用库内的一个函数只是导致了用户空间内的一个本地函数的调用，而不是系统调用。
 - 实现由操作系统直接支持的内核级的一个库。对于这种情况，库内的代码和数据结构位于内核空间。调用库中的一个API函数通常会导致对内核的系统调用。
- 目前使用的三种主要线程库是：POSIX Pthreads、Windows 和 Java：
 - Pthreads 作为 POSIX 标准的扩展，可以提供用户级或内核级的库；
 - Windows 线程库是用于 Windows 操作系统的内核级线程库；
 - Java 线程 API 允许线程在 Java 程序中直接创建和管理。

由于大多数 JVM 实例运行在宿主操作系统之上，Java 线程 API 通常采用宿主系统的线程库来实现。这意味着在 Windows 系统上，Java 线程通常采用 Windows API 来实现，而在 UNIX 和 Linux 系统中采用 Pthreads 来实现。

多线程创建策略



- 多线程创建的两个常用策略：**异步线程**和**同步线程**。
- 对异步线程，一旦父线程创建了一个子线程后，父线程就恢复自身的执行，这样父线程与子线程会并发执行。每个线程的运行独立于其他线程，父线程无需知道子线程何时终止。由于线程是独立的，所以线程之间通常很少有数据共享。如多线程 Web 服务器使用的策略就是异步线程。
- 如果父线程创建一个或多个子线程后，那么在恢复执行之前应等待所有子线程的终止（分叉-连接策略 fork-join），这就出现了同步线程。这里，由父线程创建的线程并发执行工作，但是父线程在这个工作完成之前无法继续。一旦每个线程完成了它的工作，它就会终止，并与父线程连接。只有在所有子线程都连接之后，父线程才恢复执行。
- 通常，同步线程涉及线程之间的大量数据的共享。例如，父线程可以组合由子线程计算的结果。



课后阅读

- 《操作系统概念 OSC（第9版）》第4章：4.1节 - 4.4节。
- 《现代操作系统 MOS（第4版）》第2章：2.2节。
- 《深入理解计算机系统 CSAPP（第3版）》第12章：12.3节。